

ACTIVE-IA

Devolución de Trabajo Práctico

Materia: PROG3-M2026 - Programación 3

Comisión: COMI-15 (2026)

Trabajo: Trabajo Práctico - JPA

Alumno: Macarena Marinoni

Calificación Final: 100/100

EVALUACIÓN POR CRITERIOS

C1: Estructura del modelo de dominio (UML)

12/12



La estructura del modelo de dominio está implementada de manera excelente, siguiendo fielmente el diagrama UML. Se observa el uso correcto de la superclase Base, la interfaz Calculable, los tipos de atributos, las enumeraciones y las firmas de los métodos en Pedido.

C2: Incorporación de librería Hibernate

4/4



La dependencia de Hibernate Core está correctamente incluida en el archivo build.gradle.kts, asegurando la funcionalidad JPA.

C3: Configuración de persistence.xml

8/8



El archivo persistence.xml está impecablemente configurado, incluyendo el proveedor, las clases de entidad y todas las propiedades de conexión y Hibernate según lo especificado.

C4: Mapeo JPA de Entidades Básicas

6/6



El mapeo JPA de las entidades básicas es correcto. Se utilizan adecuadamente @Entity, @MappedSuperclass, @Id, @GeneratedValue y @Enumerated(EnumType.STRING) para los enums, garantizando la persistencia de las clases y sus atributos.

C5: Relación Usuario-Pedido (asociación dirigida 1..m)

5/5



La relación Usuario-Pedido está correctamente implementada como una asociación dirigida 1..m. El Usuario tiene una colección de Pedidos (List<Pedido>) con @OneToMany y @JoinColumn, y Pedido no tiene una referencia a Usuario, lo cual es consistente con la direccionalidad.

C6: Relación Categoría-Producto (agregación 1..m)

5/5



La relación Categoría-Producto está correctamente implementada como una agregación 1..m dirigida. La Categoría posee una colección de Productos (List<Producto>) con @OneToMany y @JoinColumn, y Producto no tiene una referencia a Categoría, lo cual es consistente con la direccionalidad.

C7: Relación Pedido-DetallePedido (composición 1..m)

5/5



La relación Pedido-DetallePedido está correctamente implementada como una composición 1..m. El Pedido tiene una colección de DetallePedido (List<DetallePedido>) con @OneToMany, CascadeType.ALL y orphanRemoval = true, asegurando la vida dependiente de los detalles. La direccionalidad unidireccional es aceptable.

C8: Relación DetallePedido-Producto (asociación dirigida m..1)

5/5



La relación DetallePedido-Producto está correctamente implementada como una asociación dirigida m..1. DetallePedido tiene una referencia a Producto con @ManyToOne y @JoinColumn, y Producto no tiene una colección de DetallePedido, lo cual es consistente con la direccionalidad.

C9: CRUD: Instanciar y Persistir Entidades Base**15/15**

La instanciación y persistencia de entidades base como Producto y Categoría se realiza de forma correcta y eficiente, utilizando el patrón Builder y `em.persist()` antes de establecer relaciones complejas.

C10: CRUD: Instanciar y Persistir Pedidos con Detalles**10/10**

La instanciación y persistencia de Pedidos con sus `DetallePedido` asociados se maneja de manera excelente. El uso de `CascadeType.ALL` en las relaciones Usuario-Pedido y Pedido-DetallePedido permite una persistencia eficiente y correcta de la composición.

C11: CRUD: Actualizar Productos**10/10**

La actualización de productos se realiza correctamente utilizando `em.merge()` después de modificar los atributos de las entidades, demostrando una buena comprensión de las operaciones de actualización en JPA.

C12: CRUD: Buscar Usuario por ID**5/5**

La búsqueda de un Usuario por ID se implementa de forma óptima utilizando `em.find()`, que es el método estándar para esta operación.

C13: CRUD: Buscar Usuario por Mail**5/5**

La búsqueda de un Usuario por mail se realiza correctamente mediante una consulta JPQL con parámetros, demostrando la capacidad de realizar consultas personalizadas.

C14: CRUD: Borrar 1 Producto**5/5**

La eliminación de un producto se implementa correctamente utilizando `em.remove()`, lo cual es una forma válida de cumplir con el requisito de borrado.

FORTALEZAS

- Adherencia al UML y buenas prácticas de diseño: El modelo de dominio sigue fielmente el UML, incluyendo herencia, interfaces y métodos de dominio.
- Manejo robusto de relaciones JPA: Todas las relaciones (1..m dirigidas, composición m..1) están correctamente mapeadas con las anotaciones y cascadas adecuadas.
- Implementación completa y funcional del CRUD: Todas las operaciones CRUD solicitadas están presentes, son correctas y se ejecutan dentro de transacciones.
- Uso eficiente de Lombok y Builder: La utilización de Lombok simplifica el código y mejora la legibilidad de las entidades.

RECOMENDACIONES

1. Manejo de excepciones más granular: Aunque se incluye un try-catch general, se podría considerar un manejo de excepciones más específico para diferentes tipos de errores de persistencia.
2. Separación de lógica de negocio y persistencia: Para proyectos más grandes, se recomienda separar la lógica de persistencia (DAO/Repository) de la lógica de negocio (Service) y de la capa de presentación (Main).
3. Validación de entrada de datos: Implementar validaciones a nivel de entidad o antes de la persistencia para asegurar la integridad de los datos (ej. stock no negativo, precio > 0).
4. Uso de DTOs para la transferencia de datos: Para interacciones más complejas, considerar el uso de Data Transfer Objects (DTOs) para desacoplar las entidades de la capa de presentación o servicio.

COMENTARIOS DEL EVALUADOR

El trabajo práctico de Macarena Marinoni es sobresaliente. Demuestra una comprensión profunda de JPA, el mapeo de entidades y relaciones, y la implementación de operaciones CRUD. El código es limpio, bien estructurado y sigue las mejores prácticas de diseño.